

and JavaScript support closures, whereas C/C++ does not support them. Closures allow one way of encapsulating data with functions. In a language like C++, this is achieved by providing data members in a class, and having the class methods access these data members. The scopes of definition and of invocation remain the same. On the other hand, in JavaScript, closures are defined in one scope—say as inner functions enclosed by an outer function, and they can be invoked outside the enclosing function by being passed as a callback or return value. Let us consider the following example to clarify this:

```
function outer() {
    // Local variable that becomes part of the closure
    var count = 1000;
    var inner = function() { console.log(count); }
    return inner;
}
var myExample = outer();
myExample();
```

In the above example, there is the function 'inner', which is enclosed by the function 'outer'. The function 'inner' references the local variable 'count' of its enclosing function 'outer'. The function 'outer' returns the 'inner' function as its return value, and this is assigned to the variable 'myExample'. Now when we invoke the function 'myExample' in the last line, what is the output of the above code snippet? You will notice that the output is 1000. In the above case, though the function 'outer' has finished executing and returned, its local variable 'count' still remains defined since it is part of the closure of the 'inner' function. Hence the JavaScript runtime will not garbage-collect this as long as it is referenced by any closure. Let us consider a slight modification of the above example:

```
function outer() {
    // Local variable that becomes part of the closure
    var count = 1000;
    var inner = function() { console.log(count); }
    count++;
    return inner;
}
var myExample = outer();
myExample();
```

Now, what is the value returned by the above code snippet? Is it 1000 or 1001? Well, the answer is 1001. This essentially means that the closure variables are kept as reference and not as copy by value. Therefore, any update to these variables in their lexical scope is reflected in the closure, when the latter is invoked later. It may be useful to think of this as the JavaScript runtime retaining the enclosing

function's stack frame, as-is, without de-allocating it as long as there is a reference to it through a closure. Let us consider another example:

```
function myExample() {
    // Local variable that is part of the closure
    var count = 1000;
    // Store some references to functions as global variables
    gOutput = function() { console.log(count); }
    gSetCount = function(my_val) { count = my_val; }
}
myExample();
gOutput();
gSetCount(2000);
gOutput();
```

In the above example, we have two global variables *gOutput* and *gSetCount*, which are assigned inner functions defined inside the function 'myExample'. These inner functions access the local variable 'count' of the enclosing function 'myExample'. Hence, a closure is created for the enclosing function, and as a result both *gOutput* and *gSetCount* reference the same closure. Therefore when we invoke 'gOutput' after invoking 'gSetCount', we see that the output from 'gOutput' is 2000. Also note that each individual invocation of the enclosing function instantiates a new closure. Consider the example code given below:

```
function myExample() {
    // Local variable that is part of the closure
    var count = 1000;
    // Store some references to functions as global variables
    gOutput = function() { console.log(count); }
    gSetCount = function(my_val) { count = my_val; }
}
myExample();
gOutput();
gSetCount(2000);
gOutput();
myExample();
gOutput();
```

Now what is the last value outputted from the above code snippet? Is it 2000 or 1000? Well, it is 1000. This is because the second invocation of 'myExample' instantiates a new closure, so that when *gOutput* is called in the last line of the above snippet, it outputs 1000. It is important to understand these internal semantics of closures—else it is easy to get tripped up by them. Let us next consider how closures and loops behave.

```
function constructList(list) {
    var my_list = [];
```